CAMPUS CAREER INTELLIGENCE PLATFORM

Analytical SQL Queries & Database Reporting

Mini Project Reference Document

Technology: PostgreSQL SQL

Included: Database schema, sample data, 30 SQL queries, README outline

Purpose: analyze student profiles, placement outcomes, company hiring, salary packages, and institutional performance. Align the queries with the final database schema used by your team.

1. Database Schema

These are starter tables. If your team already has tables, use the team's existing table and column names.

1.1 Students table

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    branch VARCHAR(50) NOT NULL,
    cgpa NUMERIC(3,2),
    graduation_year INT,
    skills TEXT
);
```

1.2 Placements table

```
CREATE TABLE placements (
    placement_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES students(student_id),
    company_name VARCHAR(100),
    package_lpa NUMERIC(6,2),
    placement_status VARCHAR(20),
    placement_date DATE
);
```

2. Sample Data for Testing

Fictional records for testing only; do not present them as real institutional results.

```
INSERT INTO students
(student_name, branch, cgpa, graduation_year, skills)
VALUES
('Aman', 'IT', 8.50, 2027, 'Python, SQL'),
('Priya', 'CSE', 9.00, 2027, 'Java, Python'),
('Rahul', 'IT', 7.50, 2027, 'SQL, HTML'),
('Neha', 'ECE', 8.00, 2027, 'Python, C'),
('Rohit', 'CSE', 6.80, 2027, 'Java, SQL');

INSERT INTO placements
(student_id, company_name, package_lpa, placement_status, placement_date)
VALUES
(1, 'TCS', 5.00, 'Placed', '2026-08-10'),
(2, 'Infosys', 6.00, 'Placed', '2026-08-12'),
(3, NULL, NULL, 'Not Placed', NULL),
(4, 'Wipro', 4.50, 'Placed', '2026-08-15'),
(5, NULL, NULL, 'Not Placed', NULL);
```

3. Analytical SQL Queries (30)

Run the schema and sample-data scripts first, then execute these queries in PostgreSQL. The queries use PostgreSQL syntax such as FILTER, ILIKE, and NULLS LAST.

1. View all student records

Display all student profiles.

```
SELECT * FROM students ORDER BY student_id;
```

2. List students by branch

Show students sorted by branch and name.

```
SELECT student_id, student_name, branch, cgpa
FROM students
ORDER BY branch, student_name;
```

3. Students with CGPA 8 or above

Find students meeting a CGPA threshold.

```
SELECT student_id, student_name, branch, cgpa
FROM students
WHERE cgpa >= 8.00
ORDER BY cgpa DESC;
```

4. Count students by branch

Count student records in each branch.

```
SELECT branch, COUNT(*) AS total_students
FROM students
GROUP BY branch
ORDER BY total_students DESC;
```

5. Average CGPA by branch

Calculate average academic performance by branch.

```
SELECT branch, ROUND(AVG(cgpa), 2) AS average_cgpa
FROM students
GROUP BY branch
ORDER BY average_cgpa DESC;
```

6. Highest and lowest CGPA by branch

Compare the CGPA range across branches.

```
SELECT branch, MAX(cgpa) AS highest_cgpa, MIN(cgpa) AS lowest_cgpa
FROM students
GROUP BY branch;
```

7. Branch-wise placement rate

Count distinct students with at least one successful placement.

```
SELECT s.branch,
       COUNT(*) AS total_students,
       COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM placements p
           WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
       )) AS placed_students,
       ROUND(100.0 * COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM placements p
           WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
       )) / NULLIF(COUNT(*), 0), 2) AS placement_percentage
FROM students s
GROUP BY s.branch
ORDER BY placement_percentage DESC;
```

8. Overall placement statistics

Summarize total, placed, unplaced, and placement percentage.

```
SELECT COUNT(*) AS total_students,
       COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM placements p
           WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
       )) AS placed_students,
       COUNT(*) FILTER (WHERE NOT EXISTS (
           SELECT 1 FROM placements p
           WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
       )) AS unplaced_students,
       ROUND(100.0 * COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM placements p
           WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
       )) / NULLIF(COUNT(*), 0), 2) AS placement_percentage
FROM students s;
```

9. Average CGPA of placed students

Average CGPA for students with a successful placement.

```
SELECT ROUND(AVG(s.cgpa), 2) AS average_cgpa_of_placed_students
FROM students s
WHERE EXISTS (
    SELECT 1 FROM placements p
    WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
);
```

10. Branch-wise average CGPA of placed students

Compare CGPA among placed students by branch.

```
SELECT s.branch, COUNT(*) AS placed_students,
       ROUND(AVG(s.cgpa), 2) AS average_cgpa
FROM students s
WHERE EXISTS (
    SELECT 1 FROM placements p
    WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
)
GROUP BY s.branch
ORDER BY average_cgpa DESC;
```

11. List unplaced students

Find students without any successful placement.

```
SELECT s.student_id, s.student_name, s.branch, s.cgpa, s.skills
FROM students s
WHERE NOT EXISTS (
    SELECT 1 FROM placements p
    WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
)
ORDER BY s.cgpa DESC NULLS LAST;
```

12. Company-wise hiring count

Count placed students hired by each company.

```
SELECT company_name, COUNT(DISTINCT student_id) AS students_hired
FROM placements
WHERE placement_status = 'Placed'
GROUP BY company_name
ORDER BY students_hired DESC, company_name;
```

13. Company-wise average and highest package

Summarize package values for each company.

```
SELECT company_name,
       COUNT(DISTINCT student_id) AS students_hired,
       ROUND(AVG(package_lpa), 2) AS average_package_lpa,
       MAX(package_lpa) AS highest_package_lpa
FROM placements
WHERE placement_status = 'Placed'
GROUP BY company_name
ORDER BY highest_package_lpa DESC NULLS LAST;
```

14. Highest package by branch

Find the highest package recorded in each branch.

```
SELECT s.branch, MAX(p.package_lpa) AS highest_package_lpa
FROM students s
JOIN placements p ON p.student_id = s.student_id
WHERE p.placement_status = 'Placed'
GROUP BY s.branch
ORDER BY highest_package_lpa DESC NULLS LAST;
```

15. Average package by branch

Compare average packages across branches.

```
SELECT s.branch, ROUND(AVG(p.package_lpa), 2) AS average_package_lpa
FROM students s
JOIN placements p ON p.student_id = s.student_id
WHERE p.placement_status = 'Placed'
GROUP BY s.branch
ORDER BY average_package_lpa DESC;
```

16. Placement records by year

Count placements by calendar year.

```
SELECT EXTRACT(YEAR FROM placement_date)::INT AS placement_year,
       COUNT(DISTINCT student_id) AS students_placed
FROM placements
WHERE placement_status = 'Placed' AND placement_date IS NOT NULL
GROUP BY EXTRACT(YEAR FROM placement_date)
ORDER BY placement_year;
```

17. Recent placements

Show the most recent placement records first.

```
SELECT p.placement_id, s.student_name, s.branch,
       p.company_name, p.package_lpa, p.placement_date
FROM placements p
JOIN students s ON s.student_id = p.student_id
WHERE p.placement_status = 'Placed'
ORDER BY p.placement_date DESC NULLS LAST
LIMIT 10;
```

18. Students with multiple placement offers

Find students with multiple successful placement records.

```
SELECT s.student_id, s.student_name, COUNT(*) AS placed_records
FROM students s
JOIN placements p ON p.student_id = s.student_id
WHERE p.placement_status = 'Placed'
GROUP BY s.student_id, s.student_name
HAVING COUNT(*) > 1
ORDER BY placed_records DESC;
```

19. Students above branch average CGPA

Identify students above their branch's average CGPA.

```
SELECT s.student_id, s.student_name, s.branch, s.cgpa
FROM students s
JOIN (
    SELECT branch, AVG(cgpa) AS branch_avg_cgpa
    FROM students
    GROUP BY branch
) b ON b.branch = s.branch
WHERE s.cgpa > b.branch_avg_cgpa
ORDER BY s.branch, s.cgpa DESC;
```

20. Placement status summary

Count placement records by status.

```
SELECT placement_status, COUNT(*) AS record_count
FROM placements
GROUP BY placement_status
ORDER BY record_count DESC;
```

21. Students with missing CGPA

Find records needing academic data completion.

```
SELECT student_id, student_name, branch
FROM students
WHERE cgpa IS NULL
ORDER BY branch, student_name;
```

22. Students with missing skills

Find profiles with blank or null skill information.

```
SELECT student_id, student_name, branch
FROM students
WHERE skills IS NULL OR BTRIM(skills) = ''
ORDER BY branch, student_name;
```

23. Students above a CGPA eligibility threshold

Example screening list for CGPA 7.0 or more; actual eligibility depends on each job.

```
SELECT student_id, student_name, branch, cgpa, skills
FROM students
WHERE cgpa >= 7.00
ORDER BY cgpa DESC;
```

24. Branch-wise unplaced student count

Count students without a successful placement by branch.

```
SELECT s.branch, COUNT(*) AS unplaced_students
FROM students s
WHERE NOT EXISTS (
    SELECT 1 FROM placements p
    WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
)
GROUP BY s.branch
ORDER BY unplaced_students DESC;
```

25. Top 5 placed students by package

Rank placed students by their recorded package.

```
SELECT s.student_name, s.branch, p.company_name, p.package_lpa
FROM students s
JOIN placements p ON p.student_id = s.student_id
WHERE p.placement_status = 'Placed'
ORDER BY p.package_lpa DESC NULLS LAST
LIMIT 5;
```

26. Package distribution bands

Count placements in package ranges.

```
SELECT CASE
           WHEN package_lpa < 3 THEN 'Below 3 LPA'
           WHEN package_lpa < 6 THEN '3 to below 6 LPA'
           WHEN package_lpa < 10 THEN '6 to below 10 LPA'
           ELSE '10 LPA and above'
       END AS package_band,
       COUNT(*) AS placement_count
FROM placements
WHERE placement_status = 'Placed' AND package_lpa IS NOT NULL
GROUP BY package_band
ORDER BY MIN(package_lpa);
```

27. Branch and company hiring matrix

Show students hired by each company from each branch.

```
SELECT s.branch, p.company_name, COUNT(DISTINCT s.student_id) AS students_hired
FROM students s
JOIN placements p ON p.student_id = s.student_id
WHERE p.placement_status = 'Placed'
GROUP BY s.branch, p.company_name
ORDER BY s.branch, students_hired DESC;
```

28. Placement rate by graduation year

Calculate placement percentage for each graduating cohort.

```
SELECT s.graduation_year,
       COUNT(*) AS total_students,
       COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM placements p
           WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
       )) AS placed_students,
       ROUND(100.0 * COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM placements p
           WHERE p.student_id = s.student_id AND p.placement_status = 'Placed'
       )) / NULLIF(COUNT(*), 0), 2) AS placement_percentage
FROM students s
GROUP BY s.graduation_year
ORDER BY s.graduation_year;
```

29. Students with a particular skill keyword

Example: search student skill text for Python.

```
SELECT student_id, student_name, branch, skills
FROM students
WHERE skills ILIKE '%Python%'
ORDER BY branch, student_name;
```

30. Dashboard KPI summary

Return key figures suitable for Power BI summary cards.

```
SELECT
    (SELECT COUNT(*) FROM students) AS total_students,
    (SELECT COUNT(DISTINCT student_id) FROM placements
     WHERE placement_status = 'Placed') AS students_placed,
    (SELECT ROUND(AVG(package_lpa), 2) FROM placements
     WHERE placement_status = 'Placed') AS average_package_lpa,
    (SELECT MAX(package_lpa) FROM placements
     WHERE placement_status = 'Placed') AS highest_package_lpa;
```

4. README Documentation Outline

Project: Campus Career Intelligence Platform

Overview: A data-driven application for student profiles, placement activities, and institutional placement analytics.

Core modules: Student Profile, Placement Management, Analytics Dashboard, ML Model, Placement Prediction, Skill Gap Analysis, Job Recommendations, and role-based authentication.

Technology stack: React for frontend, FastAPI for REST APIs, PostgreSQL for data storage, SQL for analytics, Power BI for dashboards, Python for machine learning, and Git/GitHub for version control.

My contribution: Develop analytical SQL queries and prepare project documentation/README.

Setup: Create the PostgreSQL database, run the schema script, insert sample records, then execute the analytical queries.

Expected outcome: Branch-wise placement rates, placed-student CGPA analysis, company hiring summaries, package reports, unplaced-student lists, and dashboard-ready KPIs.

Important: The sample data is fictional. Replace it with the approved project schema and authorized data before official reporting.